CS 486/686
Markov Decision Processes II

Yuntian Deng

Lecture 14

RN 17.2 · PM 9.5.2–9.5.3

Reminders

> ⏰ **Assignment 1 is due TODAY** (Thu Jun 25), 11:59 PM on LEARN.
>
> 📝 Need an extension? Contact **Dake Zhang** (A1 owner) on Piazza or d346zhan@uwaterloo.ca.

Learning goals

- Trace + implement the **value iteration** algorithm for solving an MDP.
- Trace + implement the **policy iteration** algorithm for solving an MDP.

Recap: $\pi^\star = \arg\max_a Q^\star$

From L13, the optimal policy is one greedy step from $V^\star$:

$$\pi^\star(s) = \arg\max_a Q^\star(s, a).$$

Today's question

How do we actually *compute* $V^\star$?

Two algorithms: **value iteration** (iterate values) and **policy iteration** (iterate policies).

Two value functions: $V^\star$ and $Q^\star$

$V^\star(s)$ — the best expected total (discounted) reward achievable **starting from state s** and acting optimally thereafter.

$Q^\star(s, a)$ — the best expected total reward if you **start in s, take action a now**, then act optimally.

Since you would just pick the best first action:

$$V^\star(s) = \max_a Q^\star(s, a)$$

connection ①

Unrolling $Q^\star(s, a)$: reward + future value

Take action a in state s. What happens?

- You collect the reward $R(s)$.
- The environment lands you in a next state s' with probability $P(s' \mid s, a)$.
- From s' onward you act optimally — worth $V^\star(s')$.

$$
\begin{aligned}
Q^\star(s, a) &= R(s) + \mathbb{E}_{s' \sim P(\cdot \mid s, a)} \left[V^\star(s') \right] \\
&= R(s) + \sum_{s'} P(s' \mid s, a)\, V^\star(s')
\end{aligned}
$$

One fix remaining: future reward should be *discounted*.

Discounting the future

A reward received one step later is worth less than a reward now, so we **discount** the downstream value $V^\star(s')$ by $\gamma \in [0, 1)$:

$$Q^\star(s, a) = R(s) + \gamma \sum_{s'} P(s' \mid s, a)\, V^\star(s')$$

connection ②

Why discount? Future reward is less certain, and $\gamma < 1$ keeps the long-run sum finite (and the upcoming iteration convergent).

Putting it together: the Bellman equation

The two connections between $V^\star$ and $Q^\star$:

① $V^\star(s) = \max_a Q^\star(s,a)$

② $Q^\star(s,a) = R(s) + \gamma \sum_{s'} P(s' \mid s,a)\, V^\star(s')$

Substitute ② into ①:

Bellman optimality equation

$$V^\star(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s,a)\, V^\star(s')$$

$|S|$ coupled equations; $V^\star$ is their **unique** solution. Turn it into an update rule → **value iteration**.

Apply Bellman to $V^\star(s_{11})$

s_{11}	−0.04	−0.04	−0.04
−0.04	X	−0.04	−1
−0.04	−0.04	−0.04	+1

From s_{11}, each action gives a different mixture (0.8 intended + 0.1 each side):

$$
\begin{aligned}
V^\star(s_{11}) = -0.04 + \gamma\ \max\{ & \\
& 0.8\,V^\star(s_{12}) + 0.1\,V^\star(s_{21}) + 0.1\,V^\star(s_{11}), \ \texttt{// right} \\
& 0.9\,V^\star(s_{11}) + 0.1\,V^\star(s_{12}), \ \texttt{// up} \\
& 0.9\,V^\star(s_{11}) + 0.1\,V^\star(s_{21}), \ \texttt{// left} \\
& 0.8\,V^\star(s_{21}) + 0.1\,V^\star(s_{12}) + 0.1\,V^\star(s_{11}) \ \texttt{// down} \\
\}
\end{aligned}
$$

Q1. Can we solve this system of Bellman equations in polynomial time?

> **No.** The system is *nonlinear* because of the $\max$ — there is no general efficient solver. We'll use an **iterative** algorithm instead.

The value iteration algorithm

Treat the Bellman equation as an *update rule*. Let $V_i(s)$ be the i^{th} estimate.

value-iteration(MDP, ε)

1. Initialize $V_0(s)$ arbitrarily (e.g. 0).
2. For $i = 0, 1, 2, \ldots$: for each state s, apply the Bellman update

$$V_{i+1}(s) \leftarrow R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a)\, V_i(s').$$

3. Stop when $\max_s |V_{i+1}(s) - V_i(s)| < \varepsilon$.

Guarantee: $V_i \to V^\star$ as $i \to \infty$.

- **Synchronous**: keep V_i frozen while computing all of V_{i+1}.
- **Asynchronous**: update entries in place, in any order. Often converges faster.

Apply VI to the grid

Setup: $\gamma = 1, R(s) = -0.04$ for non-goal states. Terminals: $V(s_{24}) = -1, V(s_{34}) = +1$. Init $V_0(s) = 0$ for all non-terminals.

$V_0(s)$

0	0	0	0
0	X	0	−1
0	0	0	+1

After one Bellman update, only the cells adjacent to a goal will see something different from -0.04.

One iteration: $V_1(s_{23})$

s_{23} is just left of the -1 terminal. Update from V_0 (all 0 except the ± 1 terminals), $\gamma = 1$; each action is 0.8 intended + 0.1 each side (a wall bump stays put).

V_0 (input)

0	0	0	0
0	X	s_{23}	−1
0	0	0	+1

$$V_1(s_{23}) = R(s_{23}) + \gamma \max_a \sum_{s'} P(s' \mid s_{23}, a)\, V_0(s')$$

Right $0.8(-1) + 0.1(0) + 0.1(0) = -0.8$

Up $0.8(0) + 0.1(0) + 0.1(-1) = -0.1$

Down $0.8(0) + 0.1(0) + 0.1(-1) = -0.1$

Left $0.8(0) + 0.1(0) + 0.1(0) = 0$ ← best (stays clear of -1)

$$\Rightarrow V_1(s_{23}) = -0.04 + (1)(0) = -0.04$$

One iteration: $V_1(s_{33})$

s_{33} sits just left of the $+1$ terminal. Same update from $V_0, \gamma = 1$:

V_0 (input)

0	0	0	0
0	X	0	−1
0	0	s_{33}	+1

$$V_1(s_{33}) = R(s_{33}) + \gamma \max_a \sum_{s'} P(s' \mid s_{33}, a)\, V_0(s')$$

Right $0.8(+1) + 0.1(0) + 0.1(0) = 0.8$ ← best (toward $+1$)

Up $0.8(0) + 0.1(0) + 0.1(+1) = 0.1$

Down $0.8(0) + 0.1(0) + 0.1(+1) = 0.1$

Left $0.8(0) + 0.1(0) + 0.1(0) = 0$

$$\Rightarrow V_1(s_{33}) = -0.04 + (1)(0.8) = 0.76$$

The $+1$ reward has "leaked" one cell up; values propagate one cell per iteration.

Two iterations: $V_2(s)$

Apply Bellman again, using V_1 as input. For each cell we show its **best action** — the one $\max_a$ selects. Three cells worth a closer look:

$V_2(s)$

−0.08	−0.08	−0.08	−0.08
−0.08	X	0.464	−1
−0.08	0.56	0.832	+1

Q4. $V_2(s_{33})$: best action **Right** gives $0.8(+1) + 0.1(-0.04) + 0.1(0.76) = 0.872. \Rightarrow -0.04 + 0.872 = \boxed{0.832}$.

Q5. $V_2(s_{23})$: best action **Down** gives $0.8(0.76) + 0.1(-0.04) + 0.1(-1) = 0.504. \Rightarrow -0.04 + 0.504 = \boxed{0.464}$.

Q6. $V_2(s_{32})$: best action **Right** gives $0.8(0.76) + 0.1(-0.04) + 0.1(-0.04) = 0.6. \Rightarrow -0.04 + 0.6 = \boxed{0.56}$.

Values propagate one cell per iteration. After enough sweeps they converge to the L13 $V^\star$.

The value of a fixed policy V^{π}

$V^{\pi}(s)$ — the expected discounted return if you **start in s and follow the fixed policy π forever**.

With the action pinned to $\pi(s)$, it satisfies **one linear equation per state**:

$$V^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V^{\pi}(s')$$

$V^{\star}(s) = \max_{\pi} V^{\pi}(s)$. Policy iteration repeatedly computes V^{π}, then makes π greedy w.r.t. it.

Policy iteration

Insight: if one action is clearly better, you don't need precise utility values — just enough to rank actions.

policy-iteration(MDP)

1. Start from an arbitrary policy π_0.
2. Repeat:
   - **Policy evaluation:** compute $V^{\pi_i}(s)$, the utility of every state under π_i.
   - **Policy improvement:** $\pi_{i+1}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a)\, V^{\pi_i}(s')$.
3. Until $\pi_{i+1} = \pi_i$.

Terminates in finitely many steps: there are finitely many policies and each iteration strictly improves.

Policy evaluation vs Bellman: the max disappears

In PE the action is *fixed* by π, so the system becomes **linear** in V:

Policy evaluation (linear)

$$V^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V^{\pi}(s')$$

No $\max$ — the action is fixed. $|S|$ linear equations in $|S|$ unknowns.

linear: easy to solve

Bellman optimality (nonlinear)

$$V^{\star}(s) = R(s) + \gamma \max_{a} \sum_{s'} P(s' \mid s, a)\, V^{\star}(s')$$

The $\max$ makes it nonlinear — need an iterative algorithm (VI).

nonlinear: harder

Policy improvement reintroduces the $\arg\max$: $\pi_{i+1}(s) = \arg\max_{a} \sum_{s'} P(s' \mid s, a)\, V^{\pi_i}(s')$.

How do we perform policy evaluation?

Two ways to solve the linear system $V^\pi(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V^\pi(s')$:

Exactly

Linear algebra: a single solve of an $|S| \times |S|$ system.

cost: $O(|S|^3)$

Iteratively (cheap)

Run m simplified Bellman updates (no $\max$):

$$V_{j+1}(s) \leftarrow R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, V_j(s')$$

cost: $O(m \cdot |S| \cdot b)$

b = avg # of successor states (those with $P(s' \mid s, \pi(s)) > 0$). No sum over actions — π fixes the action.

In practice, a small m is usually enough — PI converges in far fewer outer iterations than VI.

Policy iteration on the grid: π_0 = all-right

Same 3×4 grid ($\gamma = 1, R = -0.04$). Start from a deliberately mediocre policy — every state goes **right**.

π_0

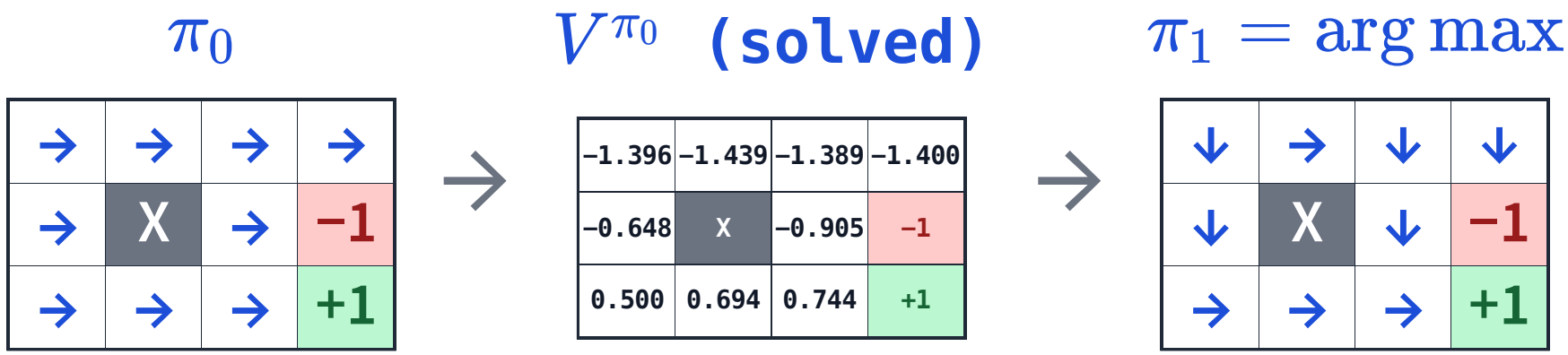
→	→	→	→
→	X	→	−1
→	→	→	+1

Policy evaluation = solve one linear equation per state. For s_{11} under π_0 (go right):

$$V^{\pi_0}(s_{11}) = -0.04 + 0.8\,V^{\pi_0}(s_{12}) + 0.1\,V^{\pi_0}(s_{11}) + 0.1\,V^{\pi_0}(s_{21})$$

9 such equations (one per non-terminal state) → hand the linear system to a solver.

Round 1: evaluate π_0, then improve



π_0 drove the top half straight into the -1 (values ≈ -1.4). Greedy improvement turns those states **down/around** toward the $+1$.

Round 2: evaluate π_1, then improve

π_1

↓	→	↓	↓
↓	X	↓	−1
→	→	→	+1

→

V^{π_1} **(solved)**

0.676	0.389	0.439	−0.885
0.762	X	0.660	−1
0.812	0.868	0.918	+1

→

$\pi_2 = \arg\max$

↓	←	↓	←
↓	X	↓	−1
→	→	→	+1

Values jump up; the top row now steers **away** from the -1. Only the corner near -1 still needs adjusting.

Convergence: PI meets VI

A couple more rounds settle the top row, and the policy stops changing — that is $\pi^\star$.

$\pi^\star$

↓	←	←	←
↓	X	↓	−1
→	→	→	+1

$V^\star = V^{\pi^\star}$

0.705	0.655	0.611	0.388
0.762	X	0.660	−1
0.812	0.868	0.918	+1

Exactly the $V^\star$ and optimal policy from L13 — **value iteration and policy iteration agree.**

Learning goals (recap)

✓ Trace and implement **value iteration** for solving an MDP.

✓ Trace and implement **policy iteration** for solving an MDP.

Next: reinforcement learning

Today we knew the dynamics P and rewards R of the MDP. What if we *don't*?

L15: learn the policy by **interacting with the environment.**